

Digital Verification – Project 3 Report

APB Bus Peripheral Verification Using UVM RAL

Coverage-Driven Register Abstraction Layer Implementation

Design Unit: AMBA APB-based Peripheral Register Bank

Specification Scope: Frontdoor/Backdoor RAL Sequences & Coverage

Student: Mostafa Mohamed Farhan

Target DUT: APB 5-Register Bank Interface

Reviewer: Eng. Hassan Khaled, Eng. Khaled Hamed, and Eng. Ali Mohamed

Contents

1	Design Architecture and Register Specification	2
1.1	Memory Map	2
1.2	Field Descriptions	2
2	UVM RAL Verification Architecture	2
3	Source Code Listings	2
3.1	Register Definitions (<code>regs.sv</code>)	2
3.2	Register Block (<code>reg_block.sv</code>)	7
3.3	Bus Adapter (<code>adapter.sv</code>)	8
3.4	Sequences	9
3.5	Environment and Test (<code>my_env.sv</code> , <code>my_test.sv</code>)	15
3.6	Interface and Top (<code>APB_If.sv</code> , <code>top.sv</code>)	17
3.7	Design Under Test (<code>APB.sv</code>)	18
3.8	Sequence Item (<code>my_sequence_item.sv</code>)	19
3.9	Sequencer (<code>my_sequencer.sv</code>)	20
3.10	Driver (<code>my_driver.sv</code>)	20
3.11	Monitor (<code>my_monitor.sv</code>)	22
3.12	Agent (<code>my_agent.sv</code>)	23
3.13	Agent Configuration (<code>my_config.sv</code>)	24
3.14	Scoreboard (<code>my_scoreboard.sv</code>)	24
3.15	Compilation File List (<code>files.txt</code>)	26
3.16	Simulation Macro Script (<code>run.do</code>)	26
4	Simulation Waveforms and Execution Logs	27
4.1	Frontdoor and Backdoor Transaction Waveforms	27
4.2	UVM Simulation Log Output	29
4.3	Functional Coverage Analysis	32

1 Design Architecture and Register Specification

The Design Under Test (DUT) is an APB-compliant peripheral housing five core registers. The lower bits are mapped to control settings, and the remaining locations operate as 32-bit data containers.

1.1 Memory Map

Address	Register	Width	Reset Value	Description
0x00	CNTRL	4 bits	0x0	Valid [3:0]. Bits [31:4] reserved (read as 0).
0x04	REG1	32 bits	0x00000000	General-purpose data input register 1.
0x08	REG2	32 bits	0x00000000	General-purpose data input register 2.
0x0C	REG3	32 bits	0x00000000	General-purpose data input register 3.
0x10	REG4	32 bits	0x00000000	General-purpose data input register 4.

Table 1: DUT Register Map Specification

1.2 Field Descriptions

Register Name: CNTRL

Offset: 0x00

Reset: 0x00000000 Width: 32-bit (Lower 4 bits active) Access: RW

Provides individual run/enable control flags for peripheral functions.

- **Bits [0]** (CTRL0): RW, Reset = 0. Control bit 0.
- **Bits [1]** (CTRL1): RW, Reset = 0. Control bit 1.
- **Bits [2]** (CTRL2): RW, Reset = 0. Control bit 2.
- **Bits [3]** (CTRL3): RW, Reset = 0. Control bit 3.
- **Bits [31:4]**: RO, Reset = 0. Reserved space, returns 0 on reads.

2 UVM RAL Verification Architecture

The verification environment leverages the standard UVM Register Abstraction Layer architecture. An adapter facilitates conversion between `uvm_reg_bus_op` and the physical `my_sequence_item` APB transfers.

- **Adapter Integration:** Translates address, data, and direction to/from APB protocol transactions.
- **Prediction Model:** Predictor updates mirrored values based on observed bus monitor transactions.
- **Stimulus Structure:** Dedicated sequence classes exercise both frontdoor and backdoor access paths.

3 Source Code Listings

3.1 Register Definitions (`regs.sv`)

```

1 package regs_pkg;
2
3     import uvm_pkg::*;
4     'include "uvm_macros.svh"
5 
```

```

6      class cntrl_reg extends uvm_reg;
7          `uvm_object_utils(cntrl_reg)
8
9          rand uvm_reg_field CTRL0;
10         rand uvm_reg_field CTRL1;
11         rand uvm_reg_field CTRL2;
12         rand uvm_reg_field CTRL3;
13         rand uvm_reg_field F_Reserved;
14
15
16         covergroup cntrl_cov;
17
18             cp_ctrl0: coverpoint CTRL0.value
19             {
20                 bins zero = {0};
21                 bins one  = {1};
22             }
23
24             cp_ctrl1: coverpoint CTRL1.value
25             {
26                 bins zero = {0};
27                 bins one  = {1};
28             }
29
30             cp_ctrl2: coverpoint CTRL2.value
31             {
32                 bins zero = {0};
33                 bins one  = {1};
34             }
35
36             cp_ctrl3: coverpoint CTRL3.value
37             {
38                 bins zero = {0};
39                 bins one  = {1};
40             }
41
42         endgroup
43
44
45         function new (string name = "cntrl_reg");
46             super.new(name,32,UVM_CVR_FIELD_VALS);
47
48             if(has_coverage(UVM_CVR_FIELD_VALS))
49                 cntrl_cov = new();
50
51         endfunction
52
53         virtual function void build();
54             CTRL0 = uvm_reg_field::type_id::create("CTRL0");
55             CTRL1 = uvm_reg_field::type_id::create("CTRL1");
56             CTRL2 = uvm_reg_field::type_id::create("CTRL2");
57             CTRL3 = uvm_reg_field::type_id::create("CTRL3");
58             F_Reserved = uvm_reg_field::type_id::create("F_Reserved");
59
60             CTRL0.configure(this,1,0,"RW",0,0,1,1,1);
61             CTRL1.configure(this,1,1,"RW",0,0,1,1,1);
62             CTRL2.configure(this,1,2,"RW",0,0,1,1,1);
63             CTRL3.configure(this,1,3,"RW",0,0,1,1,1);
64             F_Reserved.configure(this,28,4,"RO",0,0,1,1,1);
65
66         endfunction
67
68         virtual function void sample

```

```

69         (
70             uvm_reg_data_t data,
71             uvm_reg_data_t byte_en,
72             bit is_read,
73             uvm_reg_map map
74         );
75         cntrl_cov.sample();
76     endfunction
77
78 endclass
79
80 class reg1_reg extends uvm_reg;
81
82     'uvm_object_utils(reg1_reg)
83     rand uvm_reg_field data;
84
85     covergroup reg1_cov;
86         coverpoint data.value[31:0]
87         {
88             bins lower = {[0:63]};
89             bins mid   = {[64:127]};
90             bins high  = {[128:255]};
91         }
92
93     endgroup
94
95     function new(string name = "REG1");
96         super.new(name, 32, UVM_CVR_FIELD_VALS);
97
98         if(has_coverage(UVM_CVR_FIELD_VALS))
99             reg1_cov = new();
100     endfunction
101
102     virtual function void build();
103         data = uvm_reg_field::type_id::create("data");
104
105         data.configure(
106             this,
107             32,          // size
108             0,           // lsb_pos
109             "RW",        // access
110             0,           // volatile
111             32'h0,       // reset
112             1,           // has_reset
113             1,           // is_rand
114             0            // individually_accessible
115         );
116     endfunction
117
118     virtual function void sample
119     (
120         uvm_reg_data_t data,
121         uvm_reg_data_t byte_en,
122         bit is_read,
123         uvm_reg_map map
124     );
125         reg1_cov.sample();
126     endfunction
127
128 endclass
129 class reg2_reg extends uvm_reg;
130
131     'uvm_object_utils(reg2_reg)

```

```

132     rand uvm_reg_field data;
133
134     covergroup reg2_cov;
135         coverpoint data.value[31:0]
136         {
137             bins lower = {[0:63]};
138             bins mid   = {[64:127]};
139             bins high  = {[128:255]};
140         }
141
142     endgroup
143
144     function new(string name = "REG2");
145         super.new(name, 32, UVM_CVR_FIELD_VALS);
146
147         if(has_coverage(UVM_CVR_FIELD_VALS))
148             reg2_cov = new();
149     endfunction
150
151     virtual function void build();
152         data = uvm_reg_field::type_id::create("data");
153
154         data.configure(
155             this,
156             32,          // size
157             0,           // lsb_pos
158             "RW",        // access
159             0,           // volatile
160             32'h0,       // reset
161             1,           // has_reset
162             1,           // is_rand
163             0            // individually_accessible
164         );
165     endfunction
166
167     virtual function void sample
168     (
169         uvm_reg_data_t data,
170         uvm_reg_data_t byte_en,
171         bit is_read,
172         uvm_reg_map map
173     );
174     reg2_cov.sample();
175 endfunction
176
177 endclass
178
179
180 class reg3_reg extends uvm_reg;
181
182     'uvm_object_utils(reg3_reg)
183     rand uvm_reg_field data;
184
185     covergroup reg3_cov;
186         coverpoint data.value[31:0]
187         {
188             bins lower = {[0:63]};
189             bins mid   = {[64:127]};
190             bins high  = {[128:255]};
191         }
192
193     endgroup
194

```

```

195     function new(string name = "REG3");
196         super.new(name, 32, UVM_CVR_FIELD_VALS);
197
198         if(has_coverage(UVM_CVR_FIELD_VALS))
199             reg3_cov = new();
200     endfunction
201
202     virtual function void build();
203         data = uvm_reg_field::type_id::create("data");
204
205         data.configure(
206             this,
207             32,          // size
208             0,          // lsb_pos
209             "RW",        // access
210             0,          // volatile
211             32'h0,       // reset
212             1,          // has_reset
213             1,          // is_rand
214             0            // individually_accessible
215         );
216     endfunction
217
218     virtual function void sample
219     (
220         uvm_reg_data_t data,
221         uvm_reg_data_t byte_en,
222         bit is_read,
223         uvm_reg_map map
224     );
225         reg3_cov.sample();
226     endfunction
227
228 endclass
229 class reg4_reg extends uvm_reg;
230
231     'uvm_object_utils(reg4_reg)
232     rand uvm_reg_field data;
233
234     covergroup reg4_cov;
235         coverpoint data.value[31:0]
236         {
237             bins lower = {[0:63]};
238             bins mid   = {[64:127]};
239             bins high  = {[128:255]};
240         }
241
242     endgroup
243
244     function new(string name = "REG4");
245         super.new(name, 32, UVM_CVR_FIELD_VALS);
246
247         if(has_coverage(UVM_CVR_FIELD_VALS))
248             reg4_cov = new();
249     endfunction
250
251     virtual function void build();
252         data = uvm_reg_field::type_id::create("data");
253
254         data.configure(
255             this,
256             32,          // size
257             0,          // lsb_pos

```

```

258         "RW",          // access
259         0,             // volatile
260         32'h0,         // reset
261         1,             // has_reset
262         1,             // is_rand
263         0              // individually_accessible
264     );
265     endfunction
266
267     virtual function void sample
268     (
269         uvm_reg_data_t data,
270         uvm_reg_data_t byte_en,
271         bit is_read,
272         uvm_reg_map map
273     );
274         reg4_cov.sample();
275     endfunction
276
277 endclass
278
279
280 endpackage

```

Listing 1: Register Field Definitions and Covergroups

3.2 Register Block (reg_block.sv)

```

1  package reg_block_pkg;
2
3      import uvm_pkg::*;
4      import regs_pkg::*;
5      'include "uvm_macros.svh"
6
7      class reg_block extends uvm_reg_block;
8
9          'uvm_object_utils(reg_block);
10
11          rand cntrl_reg CNTRL;
12          rand reg1_reg reg1;
13          rand reg2_reg reg2;
14          rand reg3_reg reg3;
15          rand reg4_reg reg4;
16
17          function new (string name = "reg_block");
18              super.new(name, UVM_NO_COVERAGE);
19          endfunction
20
21          virtual function void build();
22              uvm_reg::include_coverage ( "*", UVM_CVR_ALL);
23
24              CNTRL = cntrl_reg::type_id::create("CNTRL");
25              reg1 = reg1_reg::type_id::create("reg1");
26              reg2 = reg2_reg::type_id::create("reg2");
27              reg3 = reg3_reg::type_id::create("reg3");
28              reg4 = reg4_reg::type_id::create("reg4");
29
30              CNTRL.configure(this); // to make reg block to be parent
31              reg1.configure(this); // to make reg block to be parent
32              reg2.configure(this); // to make reg block to be parent
33              reg3.configure(this); // to make reg block to be parent
34              reg4.configure(this); // to make reg block to be parent

```

```

35         CNTRL.build();
36         reg1.build();
37         reg2.build();
38         reg3.build();
39         reg4.build();
40
41         CNTRL.set_coverage(UVM_CVR_FIELD_VALS);
42         reg1.set_coverage(UVM_CVR_FIELD_VALS);
43         reg2.set_coverage(UVM_CVR_FIELD_VALS);
44         reg3.set_coverage(UVM_CVR_FIELD_VALS);
45         reg4.set_coverage(UVM_CVR_FIELD_VALS);
46
47         add_hdl_path("top.DUT", "RTL");
48
49         ///// For Backdoor Access /////
50         CNTRL.add_hdl_path_slice("cntrl", 0, 4);
51         reg1.add_hdl_path_slice("reg1", 0, 32);
52         reg2.add_hdl_path_slice("reg2", 0, 32);
53         reg3.add_hdl_path_slice("reg3", 0, 32);
54         reg4.add_hdl_path_slice("reg4", 0, 32);
55         ///// For Backdoor Access /////
56
57         default_map = create_map("default_map", 0, 4, UVM_LITTLE_ENDIAN);
58         default_map.add_reg(CNTRL, 32'h0, "RW");
59         default_map.add_reg(reg1, 32'h4, "RW");
60         default_map.add_reg(reg2, 32'h8, "RW");
61         default_map.add_reg(reg3, 32'hc, "RW");
62         default_map.add_reg(reg4, 32'h10, "RW");
63
64         lock_model();
65
66     endfunction
67
68 endclass
69
70 endpackage
71

```

Listing 2: RAL Block Integration and Address Mapping

3.3 Bus Adapter (adapter.sv)

```

1  package apb_adapter_pkg;
2
3      import uvm_pkg::*;
4      import my_sequence_item_pkg::*;
5      'include "uvm_macros.svh"
6      class apb_adapter extends uvm_reg_adapter;
7
8          'uvm_object_utils(apb_adapter)
9          function new(string name = "apb_adapter");
10             super.new(name);
11             supports_byte_enable = 0;
12             provides_responses    = 1;
13          endfunction
14
15          virtual function uvm_sequence_item reg2bus (const ref uvm_reg_bus_op
16              rw );
17             my_sequence_item tr;
18             tr = my_sequence_item::type_id::create("tr");
19             tr.paddr = rw.addr;
20             tr.pwdata = rw.data;

```

```

20         tr.pwrite = (rw.kind == UVM_WRITE);
21
22         return tr;
23     endfunction
24     virtual function void bus2reg (uvm_sequence_item bus_item , ref
25         uvm_reg_bus_op rw ) ;
26         my_sequence_item tr;
27         if (!$cast(tr,bus_item))
28             begin
29                 'uvm_fatal("APB_ADAPTER", "bus_item is not an apb_seq_item")
30                 return;
31             end
32
33         rw.addr = tr.paddr;
34         rw.kind = tr.pwrite ? UVM_WRITE : UVM_READ;
35         if (tr.pwrite)
36             rw.data = tr.pwdata;
37         else
38             rw.data = tr.prdata;
39
40         rw.status = UVM_IS_OK;
41     endfunction
42 endclass
43
44
45
46 endpackage

```

Listing 3: uvm_reg_adapter Implementation

3.4 Sequences

```

1  package cntrl_seq_pkg;
2
3      import uvm_pkg::*;
4      import regs_pkg::*;
5      import reg_block_pkg::*;
6      'include "uvm_macros.svh"
7
8      class cntrl_seq extends uvm_sequence;
9
10         reg_block regmodel;
11         'uvm_object_utils(cntrl_seq)
12         function new(string name = "cntrl_seq");
13             super.new(name);
14         endfunction
15
16         task body();
17             uvm_status_e status;
18             bit [31:0] rdata;
19
20             rdata = regmodel.CNTRL.get();
21             'uvm_info("CNTRL_SEQ", $sformatf("Initial Desired Value : %0d",
22                 rdata), UVM_NONE);
23
24             // Set new desired value (does NOT affect hardware yet)
25             regmodel.CNTRL.set(32'hA);
26
27             // Get updated desired value from the model (still not hardware)
28             rdata = regmodel.CNTRL.get();

```

```

28         'uvm_info("CNTRL_SEQ", $sformatf("Updated Desired Value : %0d",
29             rdata), UVM_NONE);
30
31         // Write the desired value to the actual DUT register
32         regmodel.CNTRL.update(status);
33
34         // Frontdoor Read Check
35         regmodel.CNTRL.read(status, rdata, UVM_FRONTDOOR);
36         'uvm_info("CNTRL_SEQ", $sformatf("FRONTDOOR RDATA Value : %0h",
37             rdata), UVM_NONE);
38         if (rdata != 32'hA) begin
39             'uvm_error("CNTRL_TEST", $sformatf("FRONTDOOR read mismatch!
40                 Expected: 'hA, Got: 'h%0h", rdata))
41         end else begin
42             'uvm_info("CNTRL_TEST", $sformatf("FRONTDOOR read PASSED with
43                 value: 'h%0h", rdata), UVM_NONE)
44         end
45
46         // Backdoor Access Check
47         regmodel.CNTRL.write(status, 32'h5, UVM_BACKDOOR);
48         regmodel.CNTRL.read(status, rdata, UVM_BACKDOOR);
49         'uvm_info("CNTRL_SEQ", $sformatf("BACKDOOR RDATA Value : %0h",
50             rdata), UVM_NONE);
51         if (rdata != 32'h5) begin
52             'uvm_error("CNTRL_TEST", $sformatf("BACKDOOR read mismatch!
53                 Expected: 'h5, Got: 'h%0h", rdata))
54         end else begin
55             'uvm_info("CNTRL_TEST", $sformatf("BACKDOOR read PASSED with
56                 value: 'h%0h", rdata), UVM_NONE)
57         end
58
59         // For Coverage
60         regmodel.CNTRL.write(status, 32'h0, UVM_FRONTDOOR);
61         regmodel.CNTRL.write(status, 32'hf, UVM_FRONTDOOR);
62     endtask
63
64 endclass
65
66 endpackage

```

Listing 4: cntrl_seq.sv Implementation

```

1 package reg1_seq_pkg;
2
3 import uvm_pkg::*;
4 import regs_pkg::*;
5 import reg_block_pkg::*;
6 'include "uvm_macros.svh"
7
8 class reg1_seq extends uvm_sequence;
9
10     reg_block    regmodel;
11     'uvm_object_utils(reg1_seq)
12     function new(string name = "reg1_seq");
13         super.new(name);
14     endfunction
15
16     task body();
17         uvm_status_e status;
18         bit [31:0] rdata;
19
20         rdata = regmodel.reg1.get();
21         'uvm_info("REG1_SEQ ", $sformatf("Initial Desired Value : %0d",

```

```

22         rdata), UVM_NONE);
23
24         // Set new desired value (does NOT affect hardware yet)
25         regmodel.reg1.set(32'hAA);
26
27         // Get updated desired value from the model (still not hardware)
28         rdata = regmodel.reg1.get();
29         'uvm_info("REG1_SEQ", $sformatf("Updated Desired Value : %0d",
30             rdata), UVM_NONE);
31
32         // Write the desired value to the actual DUT register
33         regmodel.reg1.update(status);
34
35         // Frontdoor Read Check
36         regmodel.reg1.read(status, rdata, UVM_FRONTDOOR);
37         'uvm_info("REG1_SEQ", $sformatf("FRONTDOOR RDATA Value : %0h",
38             rdata), UVM_NONE);
39         if (rdata != 32'hAA) begin
40             'uvm_error("REG1_TEST", $sformatf("FRONTDOOR read mismatch!
41                 Expected: 'hAA, Got: 'h%0h", rdata))
42         end else begin
43             'uvm_info("REG1_TEST", $sformatf("FRONTDOOR read PASSED with
44                 value: 'h%0h", rdata), UVM_NONE)
45         end
46
47         // Backdoor Access Check
48         regmodel.reg1.write(status, 32'h55, UVM_BACKDOOR);
49         regmodel.reg1.read(status, rdata, UVM_BACKDOOR);
50         'uvm_info("REG1_SEQ", $sformatf("BACKDOOR RDATA Value : %0h",
51             rdata), UVM_NONE);
52         if (rdata != 32'h55) begin
53             'uvm_error("REG1_TEST", $sformatf("BACKDOOR read mismatch!
54                 Expected: 'h55, Got: 'h%0h", rdata))
55         end else begin
56             'uvm_info("REG1_TEST", $sformatf("BACKDOOR read PASSED with
57                 value: 'h%0h", rdata), UVM_NONE)
58         end
59
60         //For Coverage
61         regmodel.reg1.write(status, 32'h20, UVM_FRONTDOOR); // lower
62         regmodel.reg1.write(status, 32'h55, UVM_FRONTDOOR); // mid
63         regmodel.reg1.write(status, 32'hAA, UVM_FRONTDOOR); // high
64     endtask
65
66 endclass
67
68 endpackage

```

Listing 5: reg1_seq.sv Implementation

```

1 package reg2_seq_pkg;
2
3 import uvm_pkg::*;
4 import regs_pkg::*;
5 import reg_block_pkg::*;
6 'include "uvm_macros.svh"
7
8 class reg2_seq extends uvm_sequence;
9
10     reg_block    regmodel;
11     'uvm_object_utils(reg2_seq)
12     function new(string name = "reg2_seq");
13         super.new(name);

```

```

14     endfunction
15
16     task body();
17         uvm_status_e status;
18         bit [31:0] rdata;
19
20         rdata = regmodel.reg2.get();
21         'uvm_info("REG2_SEQ", $sformatf("Initial Desired Value : %0d",
22             rdata), UVM_NONE);
23
24         // Set new desired value (does NOT affect hardware yet)
25         regmodel.reg2.set(32'hAA);
26
27         // Get updated desired value from the model (still not hardware)
28         rdata = regmodel.reg2.get();
29         'uvm_info("REG2_SEQ", $sformatf("Updated Desired Value : %0d",
30             rdata), UVM_NONE);
31
32         // Write the desired value to the actual DUT register
33         regmodel.reg2.update(status);
34
35         // Frontdoor Read Check
36         regmodel.reg2.read(status, rdata, UVM_FRONTDOOR);
37         'uvm_info("REG2_SEQ", $sformatf("FRONTDOOR RDATA Value : %0h",
38             rdata), UVM_NONE);
39         if (rdata != 32'hAA) begin
40             'uvm_error("REG2_TEST", $sformatf("FRONTDOOR read mismatch!
41                 Expected: 'hAA, Got: 'h%0h", rdata))
42         end else begin
43             'uvm_info("REG2_TEST", $sformatf("FRONTDOOR read PASSED with
44                 value: 'h%0h", rdata), UVM_NONE)
45         end
46
47         // Backdoor Access Check
48         regmodel.reg2.write(status, 32'h55, UVM_BACKDOOR);
49         regmodel.reg2.read(status, rdata, UVM_BACKDOOR);
50         'uvm_info("REG2_SEQ", $sformatf("BACKDOOR RDATA Value : %0h",
51             rdata), UVM_NONE);
52         if (rdata != 32'h55) begin
53             'uvm_error("REG2_TEST", $sformatf("BACKDOOR read mismatch!
54                 Expected: 'h55, Got: 'h%0h", rdata))
55         end else begin
56             'uvm_info("REG2_TEST", $sformatf("BACKDOOR read PASSED with
57                 value: 'h%0h", rdata), UVM_NONE)
58         end
59
60         //For Coverage
61         regmodel.reg2.write(status, 32'h20, UVM_FRONTDOOR); // lower
62         regmodel.reg2.write(status, 32'h55, UVM_FRONTDOOR); // mid
63         regmodel.reg2.write(status, 32'hAA, UVM_FRONTDOOR); // high
64
65     endtask
66
67 endclass
68
69 endpackage

```

Listing 6: reg2_seq.sv Implementation

```

1 package reg3_seq_pkg;
2
3     import uvm_pkg::*;
4     import regs_pkg::*;

```

```

5  import reg_block_pkg::*;
6  'include "uvm_macros.svh"
7
8  class reg3_seq extends uvm_sequence;
9
10     reg_block    regmodel;
11     'uvm_object_utils(reg3_seq)
12     function new(string name = "reg3_seq");
13         super.new(name);
14     endfunction
15
16     task body();
17         uvm_status_e status;
18         bit [31:0] rdata;
19
20         rdata = regmodel.reg3.get();
21         'uvm_info("REG3_SEQ", $sformatf("Initial Desired Value : %0d",
22             rdata), UVM_NONE);
23
24         // Set new desired value (does NOT affect hardware yet)
25         regmodel.reg3.set(32'hAA);
26
27         // Get updated desired value from the model (still not hardware)
28         rdata = regmodel.reg3.get();
29         'uvm_info("REG3_SEQ", $sformatf("Updated Desired Value : %0d",
30             rdata), UVM_NONE);
31
32         // Write the desired value to the actual DUT register
33         regmodel.reg3.update(status);
34
35         // Frontdoor Read Check
36         regmodel.reg3.read(status, rdata, UVM_FRONTDOOR);
37         'uvm_info("REG3_SEQ", $sformatf("FRONTDOOR RDATA Value : %0h",
38             rdata), UVM_NONE);
39         if (rdata != 32'hAA) begin
40             'uvm_error("REG3_TEST", $sformatf("FRONTDOOR read mismatch!
41                 Expected: 'hAA, Got: 'h%0h", rdata))
42         end else begin
43             'uvm_info("REG3_TEST", $sformatf("FRONTDOOR read PASSED with
44                 value: 'h%0h", rdata), UVM_NONE)
45         end
46
47         // Backdoor Access Check
48         regmodel.reg3.write(status, 32'h55, UVM_BACKDOOR);
49         regmodel.reg3.read(status, rdata, UVM_BACKDOOR);
50         'uvm_info("REG3_SEQ", $sformatf("BACKDOOR RDATA Value : %0h",
51             rdata), UVM_NONE);
52         if (rdata != 32'h55) begin
53             'uvm_error("REG3_TEST", $sformatf("BACKDOOR read mismatch!
54                 Expected: 'h55, Got: 'h%0h", rdata))
55         end else begin
56             'uvm_info("REG3_TEST", $sformatf("BACKDOOR read PASSED with
57                 value: 'h%0h", rdata), UVM_NONE)
58         end
59
60         //For Coverage
61         regmodel.reg3.write(status, 32'h20, UVM_FRONTDOOR); // lower
62         regmodel.reg3.write(status, 32'h55, UVM_FRONTDOOR); // mid
63         regmodel.reg3.write(status, 32'hAA, UVM_FRONTDOOR); // high
64     endtask
65
66 endclass

```

60 `endpackage`

Listing 7: reg3_seq.sv Implementation

```

1  package reg4_seq_pkg;
2
3      import uvm_pkg::*;
4      import regs_pkg::*;
5      import reg_block_pkg::*;
6      'include "uvm_macros.svh"
7
8      class reg4_seq extends uvm_sequence;
9
10         reg_block      regmodel;
11         'uvm_object_utils(reg4_seq)
12         function new(string name = "reg4_seq");
13             super.new(name);
14         endfunction
15
16         task body();
17             uvm_status_e status;
18             bit [31:0] rdata;
19
20             rdata = regmodel.reg4.get();
21             'uvm_info("REG4_SEQ", $sformatf("Initial Desired Value : %0d",
22                 rdata), UVM_NONE);
23
24             // Set new desired value (does NOT affect hardware yet)
25             regmodel.reg4.set(32'hAA);
26
27             // Get updated desired value from the model (still not hardware)
28             rdata = regmodel.reg4.get();
29             'uvm_info("REG4_SEQ", $sformatf("Updated Desired Value : %0d",
30                 rdata), UVM_NONE);
31
32             // Write the desired value to the actual DUT register
33             regmodel.reg4.update(status);
34
35             // Frontdoor Read Check
36             regmodel.reg4.read(status, rdata, UVM_FRONTDOOR);
37             'uvm_info("REG4_SEQ", $sformatf("FRONTDOOR RDATA Value : %0h",
38                 rdata), UVM_NONE);
39             if (rdata != 32'hAA) begin
40                 'uvm_error("REG4_TEST", $sformatf("FRONTDOOR read mismatch!
41                     Expected: 'hAA, Got: 'h%0h", rdata))
42             end else begin
43                 'uvm_info("REG4_TEST", $sformatf("FRONTDOOR read PASSED with
44                     value: 'h%0h", rdata), UVM_NONE)
45             end
46
47             // Backdoor Access Check
48             regmodel.reg4.write(status, 32'h55, UVM_BACKDOOR);
49             regmodel.reg4.read(status, rdata, UVM_BACKDOOR);
50             'uvm_info("REG4_SEQ", $sformatf("BACKDOOR RDATA Value : %0h",
51                 rdata), UVM_NONE);
52             if (rdata != 32'h55) begin
53                 'uvm_error("REG4_TEST", $sformatf("BACKDOOR read mismatch!
54                     Expected: 'h55, Got: 'h%0h", rdata))
55             end else begin
56                 'uvm_info("REG4_TEST", $sformatf("BACKDOOR read PASSED with
57                     value: 'h%0h", rdata), UVM_NONE)
58             end
59         end
60     end

```

```

52         //For Coverage
53         regmodel.reg4.write(status, 32'h20, UVM_FRONTDOOR); // lower
54         regmodel.reg4.write(status, 32'h55, UVM_FRONTDOOR); // mid
55         regmodel.reg4.write(status, 32'hAA, UVM_FRONTDOOR); // high
56     endtask
57
58 endclass
59
60 endpackage

```

Listing 8: reg4_seq.sv Implementation

3.5 Environment and Test (my_env.sv, my_test.sv)

```

1  package my_env_pkg;
2
3  import uvm_pkg::*;
4  'include "uvm_macros.svh"
5
6  import my_agent_pkg::* ;
7  import my_sequence_item_pkg::*;
8  import my_scoreboard_pkg::*;
9  import reg_block_pkg::*;
10 import apb_adapter_pkg::*;
11
12
13 'define create(type , inst_name)  type::type_id::create(inst_name,this);
    // CREATE() HERE REPLACES THE CONSTRUCTION LINE LARGE CODE LIKE A TEXT
    REPLACEMENT
14
15 class my_env extends uvm_env;
16     'uvm_component_utils(my_env)
17
18
19     my_agent    agent;
20     my_scoreboard sco;
21     reg_block   regmodel;
22     uvm_reg_predictor #(my_sequence_item) predictor;
23     apb_adapter adapter;
24
25
26     function new (string name = "my_env" , uvm_component parent = null);
27         super.new(name , parent);
28     endfunction
29
30
31     function void build_phase(uvm_phase phase);
32         super.build_phase(phase);
33         'uvm_info("MY_ENV" , "ENVIRONMENT BUILT" , UVM_LOW);
34         agent      = 'create(my_agent,"agent");           //HERE WE USED THE MACRO
                    //INSTEAD OF LARGE CODE WHICH DEFINED ABOVE
35         sco        = 'create(my_scoreboard,"sco");
36         regmodel   = 'create(reg_block,"regmodel");
37         adapter    = 'create(apb_adapter,"adapter");
38         predictor  = uvm_reg_predictor#(my_sequence_item)::type_id::create("
                    predictor", this);
39         regmodel.build();
40         regmodel.lock_model();
41
42     endfunction
43
44

```

```

45     function void connect_phase(uvm_phase phase);
46         super.connect_phase(phase);
47
48         regmodel.default_map.set_sequencer(agent.seqr, adapter);
49         agent.mon.mon_ap.connect(predictor.bus_in);
50         predictor.map = regmodel.default_map;
51         predictor.adapter = adapter;
52
53         agent.mon.mon_ap.connect(sco.sco_ap);
54     endfunction
55
56 endclass
57
58
59 endpackage

```

Listing 9: my_env.sv RAL Integration

```

1  package my_test_pkg;
2
3  import uvm_pkg::*;
4  import cntrl_seq_pkg::*;
5  import reg1_seq_pkg::*;
6  import reg2_seq_pkg::*;
7  import reg3_seq_pkg::*;
8  import reg4_seq_pkg::*;
9  'include "uvm_macros.svh"
10
11 import my_env_pkg::*;
12
13
14 'define create(type , inst_name)  type::type_id::create(inst_name,this); //
    This is a macro to save the much code into small one for constructing
15
16
17 class my_test extends uvm_test;
18     'uvm_component_utils(my_test)
19
20
21     my_env  env;
22
23
24     function new (string name = "my_test" , uvm_component parent = null);
25         super.new(name , parent);
26     endfunction
27
28
29     function void build_phase(uvm_phase phase);
30         super.build_phase(phase);
31
32         env = 'create(my_env , "env");
33         'uvm_info("MY_TEST" , "TEST BUILT" , UVM_LOW);
34
35     endfunction
36
37
38
39
40     function void end_of_elaboration_phase(uvm_phase phase);
41         super.end_of_elaboration_phase(phase);
42         uvm_top.print_topology();
43     endfunction
44

```

```

45
46 task run_phase(uvm_phase phase);
47
48     cntrl_seq ctrl_seq;
49     reg1_seq  reg1_seq_h;
50     reg2_seq  reg2_seq_h;
51     reg3_seq  reg3_seq_h;
52     reg4_seq  reg4_seq_h;
53
54     phase.raise_objection(this);
55
56     ctrl_seq = cntrl_seq::type_id::create("ctrl_seq");
57     reg1_seq_h = reg1_seq::type_id::create("reg1_seq_h");
58     reg2_seq_h = reg2_seq::type_id::create("reg2_seq_h");
59     reg3_seq_h = reg3_seq::type_id::create("reg3_seq_h");
60     reg4_seq_h = reg4_seq::type_id::create("reg4_seq_h");
61
62     // Give the RAL sequence access to the register model
63     ctrl_seq.regmodel = env.regmodel;
64     `uvm_info("CNTRL RUN TEST", "TEST HERE", UVM_LOW)
65     ctrl_seq.start(null);
66
67     reg1_seq_h.regmodel = env.regmodel;
68     `uvm_info("REG 1 RUN TEST", "TEST HERE", UVM_LOW)
69     reg1_seq_h.start(null);
70
71     reg2_seq_h.regmodel = env.regmodel;
72     `uvm_info("REG 2 RUN TEST", "TEST HERE", UVM_LOW)
73     reg2_seq_h.start(null);
74
75     reg3_seq_h.regmodel = env.regmodel;
76     `uvm_info("REG 3 RUN TEST", "TEST HERE", UVM_LOW)
77     reg3_seq_h.start(null);
78
79     reg4_seq_h.regmodel = env.regmodel;
80     `uvm_info("REG 4 RUN TEST", "TEST HERE", UVM_LOW)
81     reg4_seq_h.start(null);
82
83
84     phase.drop_objection(this);
85
86 endtask
87
88 endclass
89
90 endpackage

```

Listing 10: my_test.sv Sequence Execution Flow

3.6 Interface and Top (APB_If.sv, top.sv)

```

1
2 interface APB_If;
3
4     logic    [31 : 0]    paddr   ;
5     logic    [31 : 0]    pwrite  ;
6     logic    [31 : 0]    prdata  ;
7     logic    [31 : 0]    pwrite  ;
8     logic    [31 : 0]    pwrite  ;
9     logic    [31 : 0]    pwrite  ;
10    logic    [31 : 0]    pwrite  ;
11    logic    [31 : 0]    pwrite  ;

```

```

12
13 endinterface

```

Listing 11: APB_If.sv Protocol Interface

```

1  import uvm_pkg::*;
2  `include "uvm_macros.svh"
3
4  import my_test_pkg::*;
5  `include "APB_If.sv"
6
7
8  module top;
9
10 APB_If APB_vif();
11
12 APB DUT(.paddr  (APB_vif.paddr)    ,
13         .pwrite (APB_vif.pwrite)  ,
14         .paddr  (APB_vif.paddr)    ,
15         .pwrite (APB_vif.pwrite)  ,
16         .psel   (APB_vif.psel)    ,
17         .penable(APB_vif.penable)  ,
18         .presetn(APB_vif.presetn)  ,
19         .pclk    (APB_vif.pclk)    );
20
21
22
23 always #10 APB_vif.pclk = ~APB_vif.pclk;
24
25
26 initial begin
27     APB_vif.pclk = 0;
28     uvm_config_db#(virtual APB_If)::set(null, "*", "APB_vif",APB_vif);
29     run_test("my_test");
30 end
31
32 endmodule

```

Listing 12: top.sv Testbench Top

3.7 Design Under Test (APB.sv)

```

1  module APB //ADVANCED PERIPHERAL
2      BUS
3  (
4      input                pclk,
5      input                presetn,
6      input [31 : 0]       paddr,
7      input [31 : 0]       pwrite,
8      input                psel,
9      input                penable,
10     output [31 : 0]       prdata
11 );
12
13 reg [31:0]  cntrl = 0; /// cntrl : [reg4 reg3 reg2 reg1] //Edited
14 reg [31:0]  reg1  = 0; /// datainput 1
15 reg [31:0]  reg2  = 0; /// datainput 2
16 reg [31:0]  reg3  = 0; /// datainput 3
17 reg [31:0]  reg4  = 0; /// datainput 4
18
19

```

```

20  reg      [31 : 0]      rdata_tmp = 0;
21
22
23
24  // Set all registers to default values
25  always @ (posedge pclk)
26  begin
27      if( !presetn )
28      begin
29          cntrl      <= 32'b0000;
30          reg1       <= 32'h00000000;
31          reg2       <= 32'h00000000;
32          reg3       <= 32'h00000000;
33          reg4       <= 32'h00000000;
34          rdata_tmp  <= 32'h00000000;
35      end
36      //update values of register
37      else if( psel && penable && pwrite )
38      begin
39          case( paddr )
40              'h0      : cntrl <= pwrite;
41              'h4      : reg1  <= pwrite;
42              'h8      : reg2  <= pwrite;
43              'hc      : reg3  <= pwrite;
44              'h10     : reg4  <= pwrite;
45          endcase
46      end
47      else if (psel && penable && !pwrite )
48      begin
49          case( paddr )
50              'h0      : rdata_tmp <= {28'h00000000,cntrl};
51              'h4      : rdata_tmp <= reg1;
52              'h8      : rdata_tmp <= reg2;
53              'hc      : rdata_tmp <= reg3;
54              'h10     : rdata_tmp <= reg4;
55              default : rdata_tmp <= 32'h00000000;
56          endcase
57      end
58  end
59
60  end
61
62
63  assign prdata =  rdata_tmp;
64
65  endmodule

```

Listing 13: APB.sv Core Peripheral RTL Implementation

3.8 Sequence Item (my_sequence_item.sv)

```

1  package my_sequence_item_pkg;
2
3  import uvm_pkg::*;
4  'include "uvm_macros.svh"
5
6
7  class my_sequence_item extends uvm_sequence_item;
8      'uvm_object_utils(my_sequence_item)
9
10     rand logic [31 : 0]      paddr ;
11     rand logic [31 : 0]      pwrite ;

```

```

12     logic    [31 : 0]    prdata ;
13     rand logic          pwrite ;
14     bit      psel      ;
15     bit      penable;
16
17
18     constraint paddr_c {
19         paddr inside {'h0 , 'h4 , 'hC , 'h10};
20     }
21
22
23     function new (string name = "my_sequence_item");
24         super.new(name);
25     endfunction
26
27 endclass
28
29 endpackage

```

Listing 14: my_sequence_item.sv APB Bus Transaction Definition

3.9 Sequencer (my_sequencer.sv)

```

1  package my_sequencer_pkg;
2
3  import uvm_pkg::*;
4  'include "uvm_macros.svh"
5
6  import my_sequence_item_pkg::*;
7
8
9  class my_sequencer extends uvm_sequencer #(my_sequence_item);
10     'uvm_component_utils(my_sequencer)
11
12     function new (string name = "my_sequencer" , uvm_component parent = null);
13         super.new(name , parent);
14     endfunction
15
16
17     function void build_phase(uvm_phase phase);
18         super.build_phase(phase);
19         'uvm_info("MY_SEQUENCER" , "SEQUENCER BUILT" , UVM_LOW);
20     endfunction
21
22 endclass
23
24
25 endpackage

```

Listing 15: my_sequencer.sv APB Sequencer Component

3.10 Driver (my_driver.sv)

```

1  package my_driver_pkg;
2
3  import uvm_pkg::*;
4  'include "uvm_macros.svh"
5
6  import my_sequence_item_pkg::*;
7

```

```

8
9 class my_driver extends uvm_driver #(my_sequence_item);
10   'uvm_component_utils(my_driver)
11
12   my_sequence_item req;
13   virtual APB_If APB_vif;
14
15   function new(string name = "my_driver" , uvm_component parent = null);
16     super.new(name , parent);
17   endfunction
18
19
20   function void build_phase(uvm_phase phase);
21     super.build_phase(phase);
22
23     'uvm_info("MY_DRIVER" , "DRIVER BUILT" , UVM_LOW);
24     req = my_sequence_item::type_id::create("req" , this);
25
26     if(!(uvm_config_db#(virtual APB_If)::get(this, "", "APB_vif", APB_vif)
27       )) begin
28       'uvm_fatal("DRIVER" , "FAILED GETTING INTERFACE");
29     end
30   endfunction
31
32
33
34   virtual task run_phase (uvm_phase phase);
35     bit [31:0] data;
36     APB_vif.presetn <= 1'b1;
37     APB_vif.psel <= 0;
38     APB_vif.penable <= 0;
39     APB_vif.pwrite <= 0;
40     APB_vif.paddr <= 0;
41     APB_vif.pwdata <= 0;
42     forever begin
43       seq_item_port.get_next_item (req);
44       if (req.pwrite)
45         begin
46           write();
47         end
48       else
49         begin
50           read();
51         end
52       seq_item_port.item_done (req);
53     end
54   endtask
55
56   //////////////////////////////////////
57
58
59   //write data to dut -> psel -> pen
60
61   virtual task write();
62     @(posedge APB_vif.pclk);
63     APB_vif.paddr <= req.paddr;
64     APB_vif.pwdata <= req.pwdata;
65     APB_vif.pwrite <= 1'b1;
66     APB_vif.psel <= 1'b1;
67     @(posedge APB_vif.pclk);
68     APB_vif.penable <= 1'b1;
69

```

```

70     'uvm_info("DRV", $sformatf("Mode : Write WDATA : %0d ADDR : %0d",
71         APB_vif.pwdata, APB_vif.paddr), UVM_NONE);
72     @(posedge APB_vif.pclk);
73     APB_vif.psel     <= 1'b0;
74     APB_vif.penable <= 1'b0;
75     endtask
76
77     ////read data from dut
78     virtual task read();
79         @(posedge APB_vif.pclk);
80         APB_vif.paddr     <= req.paddr;
81         APB_vif.pwrite    <= 1'b0;
82         APB_vif.psel     <= 1'b1;
83         @(posedge APB_vif.pclk);
84         APB_vif.penable <= 1'b1;
85         'uvm_info("DRV", $sformatf("Mode : Write WDATA : %0d ADDR : %0d RDATA
86             : %0d", APB_vif.pwdata, APB_vif.paddr, APB_vif.prdata), UVM_NONE
87             );
88
89         @(posedge APB_vif.pclk);
90         #1; //ADDED
91         APB_vif.psel     <= 1'b0;
92         APB_vif.penable <= 1'b0;
93         req.prdata      = APB_vif.prdata;
94     endtask
95
96
97 endclass
98 endpackage

```

Listing 16: my_driver.sv APB Protocol Bus Driver

3.11 Monitor (my_monitor.sv)

```

1  package my_monitor_pkg;
2
3  import uvm_pkg::*;
4  'include "uvm_macros.svh"
5
6  import my_sequence_item_pkg::*;
7
8
9  class my_monitor extends uvm_monitor;
10     'uvm_component_utils(my_monitor)
11
12     virtual APB_If APB_vif;
13     my_sequence_item seq_item;
14     uvm_analysis_port #(my_sequence_item) mon_ap;
15
16     function new (string name = "my_monitor" , uvm_component parent = null);
17         super.new(name , parent);
18         mon_ap = new("mon_ap" , this);
19     endfunction
20
21
22     function void build_phase(uvm_phase phase);
23         super.build_phase(phase);
24         'uvm_info("MY_MONITOR" , "MONITOR BUILT" , UVM_LOW);
25

```

```

26         if(!(uvm_config_db#(virtual APB_If)::get(this, "", "APB_vif", APB_vif)
27             )) begin
28             'uvm_fatal("MON" , "FAILED GETTING INTERFACE");
29         end
30     endfunction
31
32
33     task run_phase(uvm_phase phase);
34
35
36         forever begin
37             my_sequence_item seq_item = my_sequence_item::type_id::create(
38                 "seq_item");
39             @(posedge APB_vif.pclk);
40             seq_item.paddr = APB_vif.paddr;
41             seq_item.pwdata = APB_vif.pwdata;
42             seq_item.pwrite = APB_vif.pwrite;
43             seq_item.psel = APB_vif.psel;
44             seq_item.penable = APB_vif.penable;
45             //@(posedge APB_vif.pclk); //ADDED
46             #1; //ADDED
47             seq_item.prdata = APB_vif.prdata;
48             mon_ap.write(seq_item);
49         end
50
51     endtask
52
53 endclass
54
55
56 endpackage

```

Listing 17: my_monitor.sv APB Protocol Bus Monitor & Analysis Port

3.12 Agent (my_agent.sv)

```

1 package my_agent_pkg;
2
3 import uvm_pkg::*;
4 'include "uvm_macros.svh"
5
6 import my_sequencer_pkg::*;
7 import my_driver_pkg::*;
8 import my_monitor_pkg::*;
9
10 'define create(type , inst_name) type::type_id::create(inst_name, this);
11
12 class my_agent extends uvm_agent;
13     'uvm_component_utils(my_agent)
14
15
16     my_driver driver;
17     my_sequencer seqr;
18     my_monitor mon;
19
20
21     function new (string name = "my_agent" , uvm_component parent = null);
22         super.new(name , parent);
23     endfunction
24

```

```

25
26 function void build_phase(uvm_phase phase);
27     super.build_phase(phase);
28
29     'uvm_info("MY_AGENT" , "AGENT BUILT" , UVM_LOW);
30
31     driver = 'create(my_driver , "driver");
32     seqr   = 'create(my_sequencer, "seqr");
33     mon    = 'create(my_monitor, "mon");
34
35 endfunction
36
37
38 function void connect_phase(uvm_phase phase);
39     super.connect_phase(phase);
40     driver.seq_item_port.connect(seqr.seq_item_export);
41 endfunction
42
43
44
45 endclass
46 endpackage

```

Listing 18: my_agent.sv UVC Agent Encapsulation

3.13 Agent Configuration (my_config.sv)

```

1 package my_config_pkg;
2
3 import uvm_pkg::*;
4 'include "uvm_macros.svh"
5
6
7 class my_config extends uvm_object;
8
9     'uvm_object_utils(my_config);
10
11 function new(string name = "my_config");
12     super.new(name);
13 endfunction
14
15 endclass
16
17 endpackage

```

Listing 19: my_config.sv Configuration Object

3.14 Scoreboard (my_scoreboard.sv)

```

1 package my_scoreboard_pkg;
2
3 import uvm_pkg::*;
4 'include "uvm_macros.svh"
5
6 import my_sequence_item_pkg::*;
7
8
9 class my_scoreboard extends uvm_scoreboard;
10     'uvm_component_utils(my_scoreboard)
11

```

```

12 virtual APB_If APB_vif;
13 my_sequence_item seq_item;
14 uvm_analysis_export #(my_sequence_item) sco_ap;
15 uvm_tlm_analysis_fifo #(my_sequence_item) sb_fifo;
16
17 logic [31:0] Assoc_Array [bit[31:0]] ;
18
19 function new (string name = "my_scoreboard" , uvm_component parent = null);
20     super.new(name , parent);
21     sco_ap = new("sco_ap" , this);
22     seq_item = new("seq_item");
23     sb_fifo = new("sb_fifo",this);
24 endfunction
25
26
27 function void build_phase(uvm_phase phase);
28     super.build_phase(phase);
29     'uvm_info("my_scoreboard" , "SCO BUILT" , UVM_LOW);
30
31     if(!(uvm_config_db#(virtual APB_If)::get(this, "", "APB_vif", APB_vif)
32     )) begin
33         'uvm_fatal("SCO" , "FAILED GETTING INTERFACE");
34     end
35 endfunction
36
37 function void connect_phase(uvm_phase phase);
38     super.connect_phase(phase);
39     sco_ap.connect(sb_fifo.analysis_export);
40 endfunction
41
42
43
44 task run_phase(uvm_phase phase);
45     forever begin
46         sb_fifo.get(seq_item);
47         //@(posedge APB_vif.pclk);
48         if(seq_item.psel && seq_item.penable && APB_vif.presetn) begin
49             if(seq_item.pwrite) begin
50                 Assoc_Array[seq_item.paddr] = seq_item.pwdata;
51                 'uvm_info("my_scoreboard" , "SCO Write DATA" , UVM_LOW);
52             end
53             else begin
54                 if(seq_item.prdata == Assoc_Array[seq_item.paddr]) begin
55                     'uvm_info("RIGHT DATA" , $sformatf("rdata = %0h ,
56                         Expec_rdata = %0h" , seq_item.prdata, Assoc_Array[
57                             seq_item.paddr]) , UVM_LOW);
58                 end
59                 else begin
60                     'uvm_error("WRONG DATA", $sformatf("rdata = %0h ,
61                         Expec_rdata = %0h" , seq_item.prdata, Assoc_Array[
62                             seq_item.paddr]));
63                     end
64                 end
65             end
66         end
67     endtask
68 endclass
69
70 endpackage

```

Listing 20: my_scoreboard.sv Data Integrity and Prediction Checking

3.15 Compilation File List (files.txt)

```
1 adapter.sv
2 APB.sv
3 APB_If.sv
4 my_agent.sv
5 my_config.sv
6 my_driver.sv
7 my_env.sv
8 my_monitor.sv
9 my_scoreboard.sv
10 my_sequence_item.sv
11 my_sequencer.sv
12 reg_block.sv
13 regs.sv
14 cntrl_seq.sv
15 reg1_seq.sv
16 reg2_seq.sv
17 reg3_seq.sv
18 reg4_seq.sv
19 my_test.sv
20 top.sv
```

Listing 21: files.txt Project Package & File Order

3.16 Simulation Macro Script (run.do)

```
1 vlib work
2 vlog -f files.txt +cover
3 vsim -voptargs=+acc work.top -cover
4 run 0
5 do wave.do
6 run -all
```

Listing 22: run.do QuestaSim Compilation & Execution Script

4 Simulation Waveforms and Execution Logs

4.1 Frontdoor and Backdoor Transaction Waveforms

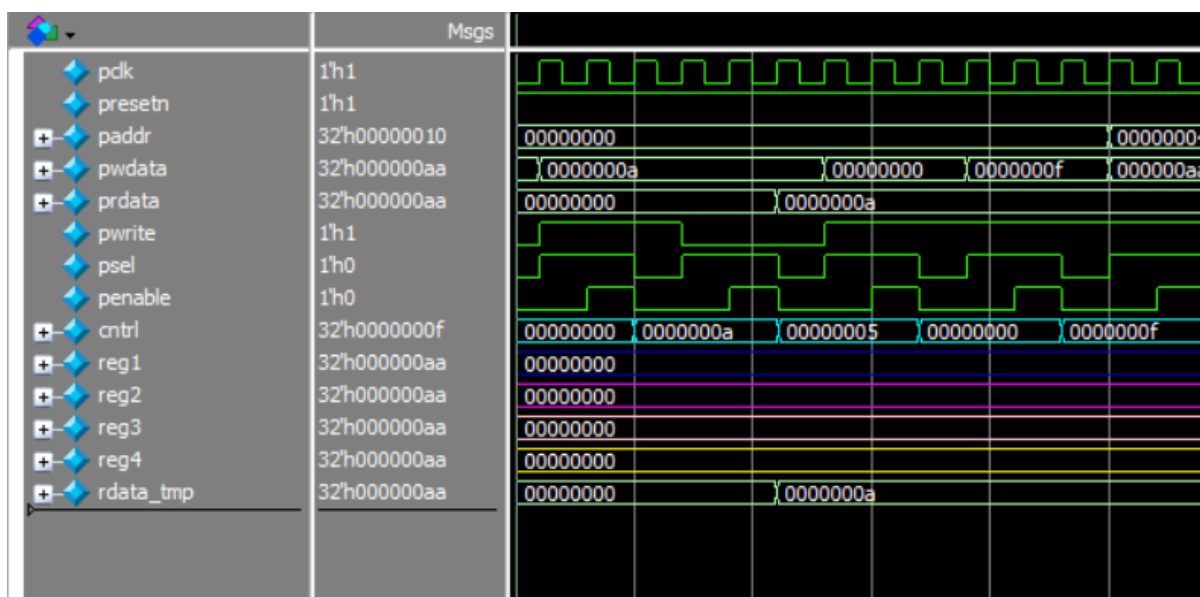


Figure 1: Waveform 1: CNTRL Register Frontdoor Write/Read and Backdoor Verification

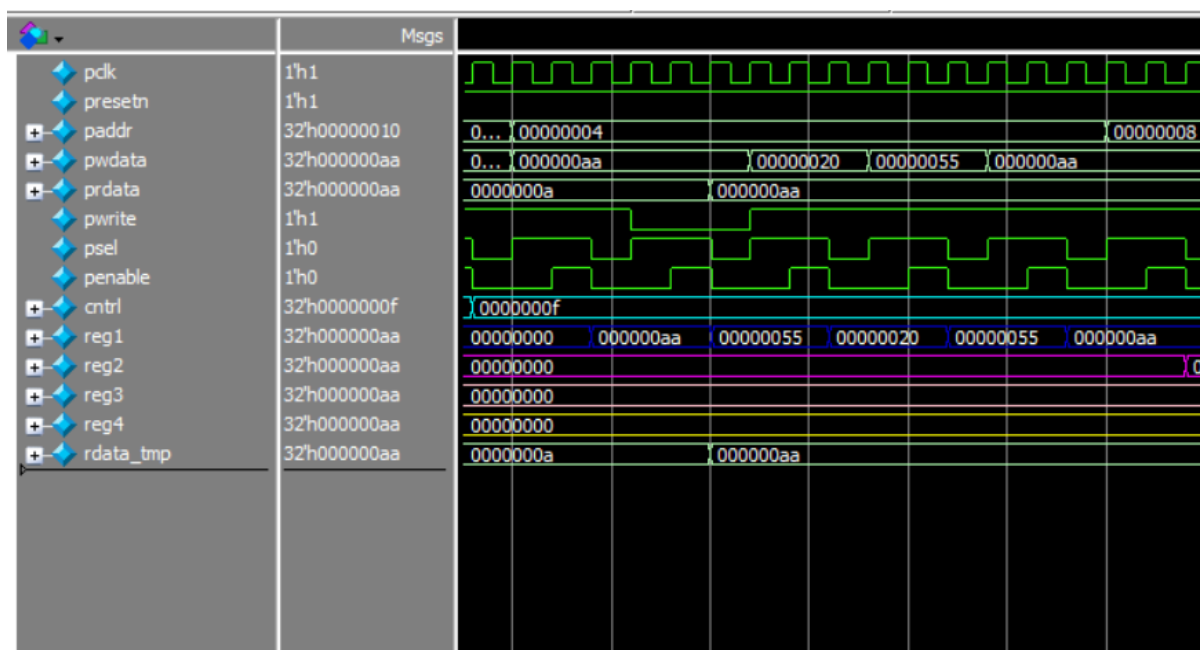


Figure 2: Waveform 2: REG1 Register Frontdoor Write/Read and Backdoor Verification

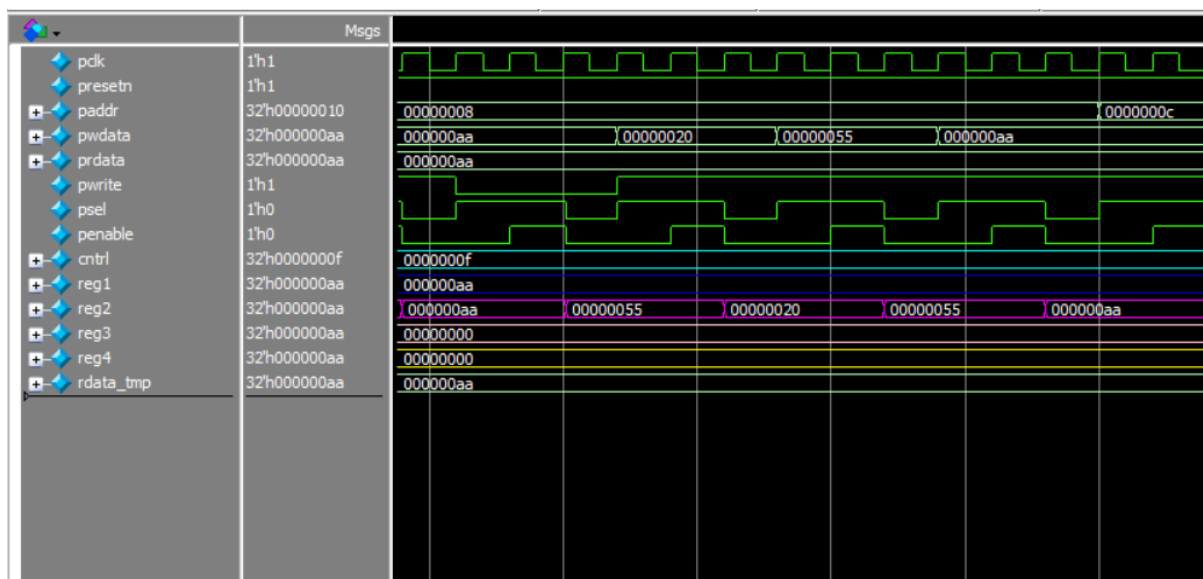


Figure 3: Waveform 3: REG2 Register Frontdoor Write/Read and Backdoor Verification

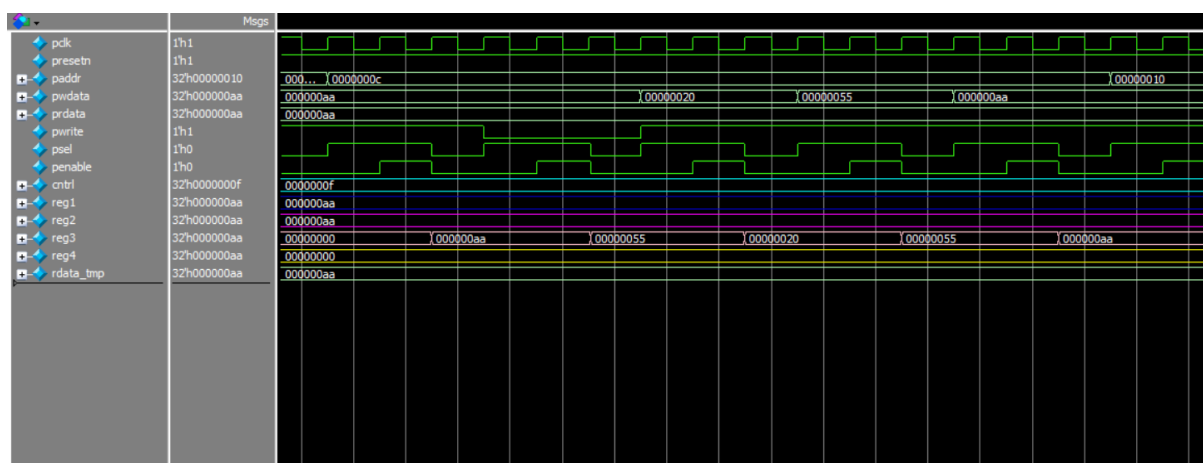


Figure 4: Waveform 4: REG3 Register Frontdoor Write/Read and Backdoor Verification

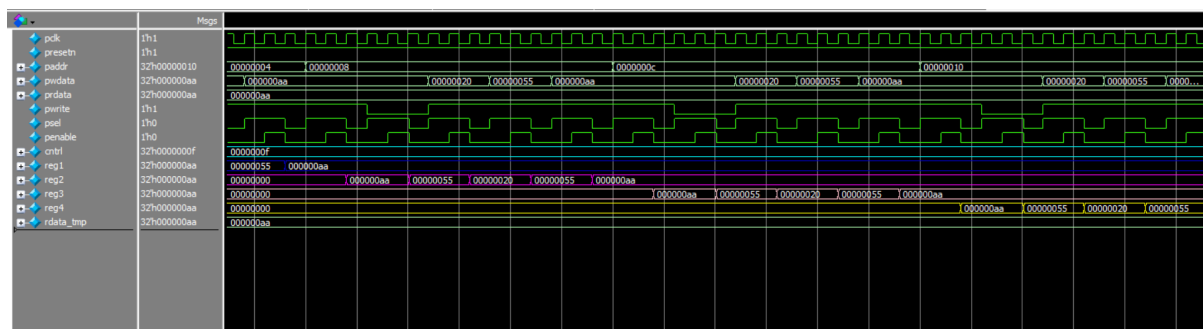


Figure 5: Waveform 5: Full Consecutive Execution of all 5 RAL Sequences Across the Simulation Horizon

4.2 UVM Simulation Log Output

```

#
# (Specify +UVM_NO_RELNOTES to turn off this notice)
#
# UVM_INFO verillog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verillog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM] questa_uvm::init(+struct)
# UVM_INFO @ 0: reporter [RNTST] Running test my_test...
# UVM_INFO my_test.sv(33) @ 0: uvm_test_top [MY_TEST] TEST BUILT
# UVM_INFO my_env.sv(33) @ 0: uvm_test_top.env [MY_ENV] ENVIRONMENT BUILT
# UVM_INFO my_agent.sv(29) @ 0: uvm_test_top.env.agent [MY_AGENT] AGENT BUILT
# UVM_INFO my_driver.sv(23) @ 0: uvm_test_top.env.agent.driver [MY_DRIVER] DRIVER BUILT
# UVM_INFO my_monitor.sv(24) @ 0: uvm_test_top.env.agent.mon [MY_MONITOR] MONITOR BUILT
# UVM_INFO my_sequencer.sv(19) @ 0: uvm_test_top.env.agent.seqr [MY_SEQUENCER] SEQUENCER BUILT
# UVM_INFO my_scoreboard.sv(29) @ 0: uvm_test_top.env.sco [my_scoreboard] SCO BUILT
# UVM_INFO @ 0: reporter [UVMTOP] UVM testbench topology:

```

Figure 6: Execution Log 0: Testbench Build Phase and Component Construction

```

#
# UVM_INFO my_test.sv(64) @ 0: uvm_test_top [CNTRL RUN TEST] TEST HERE
# UVM_INFO cntrl_seq.sv(21) @ 0: reporter@@cntrl_seq [CNTRL SEQ] Initial Desired Value : 0
# UVM_INFO cntrl_seq.sv(28) @ 0: reporter@@cntrl_seq [CNTRL SEQ] Updated Desired Value : 10
# UVM_INFO my_driver.sv(70) @ 30: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 10 ADDR : 0
# UVM_INFO my_scoreboard.sv(51) @ 51: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(84) @ 90: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 10 ADDR : 0 RDATA : 0
# UVM_INFO my_scoreboard.sv(55) @ 111: uvm_test_top.env.sco [RIGHT DATA] rdata = a , Expec_rdata = a
# UVM_INFO cntrl_seq.sv(35) @ 111: reporter@@cntrl_seq [CNTRL SEQ] FRONTDOOR RDATA Value : a
# UVM_INFO cntrl_seq.sv(39) @ 111: reporter@@cntrl_seq [CNTRL TEST] FRONTDOOR read PASSED with value: 'ha
# UVM_INFO cntrl_seq.sv(45) @ 111: reporter@@cntrl_seq [CNTRL SEQ] BACKDOOR RDATA Value : 5
# UVM_INFO cntrl_seq.sv(49) @ 111: reporter@@cntrl_seq [CNTRL TEST] BACKDOOR read PASSED with value: 'h5
# UVM_INFO my_driver.sv(70) @ 150: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 0 ADDR : 0
# UVM_INFO my_scoreboard.sv(51) @ 171: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 210: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 15 ADDR : 0

```

Figure 7: Execution Log 1: CNTRL Sequence Terminal Output

```

# UVM_INFO my_test.sv(68) @ 230: uvm_test_top [REG 1 RUN TEST] TEST HERE
# UVM_INFO reg1_seq.sv(21) @ 230: reporter@@reg1_seq_h [REG1 SEQ ] Initial Desired Value : 0
# UVM_INFO reg1_seq.sv(28) @ 230: reporter@@reg1_seq_h [REG1 SEQ] Updated Desired Value : 170
# UVM_INFO my_scoreboard.sv(51) @ 231: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 270: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 4
# UVM_INFO my_scoreboard.sv(51) @ 291: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(84) @ 330: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 4 RDATA : 10
# UVM_INFO my_scoreboard.sv(55) @ 351: uvm_test_top.env.sco [RIGHT DATA] rdata = aa , Expec_rdata = aa
# UVM_INFO reg1_seq.sv(35) @ 351: reporter@@reg1_seq_h [REG1 SEQ] FRONTDOOR RDATA Value : aa
# UVM_INFO reg1_seq.sv(39) @ 351: reporter@@reg1_seq_h [REG1 TEST] FRONTDOOR read PASSED with value: 'haa
# UVM_INFO reg1_seq.sv(45) @ 351: reporter@@reg1_seq_h [REG1 SEQ] BACKDOOR RDATA Value : 55
# UVM_INFO reg1_seq.sv(49) @ 351: reporter@@reg1_seq_h [REG1 TEST] BACKDOOR read PASSED with value: 'h55
# UVM_INFO my_driver.sv(70) @ 390: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 32 ADDR : 4
# UVM_INFO my_scoreboard.sv(51) @ 411: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 450: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 85 ADDR : 4
# UVM_INFO my_scoreboard.sv(51) @ 471: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 510: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 4

```

Figure 8: Execution Log 2: REG1 Sequence Terminal Output

```

# UVM_INFO my_test.sv(72) @ 530: uvm_test_top [REG 2 RUN TEST] TEST HERE
# UVM_INFO reg2_seq.sv(21) @ 530: reporter@@reg2_seq_h [REG2 SEQ ] Initial Desired Value : 0
# UVM_INFO reg2_seq.sv(28) @ 530: reporter@@reg2_seq_h [REG2 SEQ] Updated Desired Value : 170
# UVM_INFO my_scoreboard.sv(51) @ 531: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 570: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 8
# UVM_INFO my_scoreboard.sv(51) @ 591: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(84) @ 630: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 8 RDATA : 170
# UVM_INFO my_scoreboard.sv(55) @ 651: uvm_test_top.env.sco [RIGHT DATA] rdata = aa , Expec_rdata = aa
# UVM_INFO reg2_seq.sv(35) @ 651: reporter@@reg2_seq_h [REG2 SEQ] FRONTDOOR RDATA Value : aa
# UVM_INFO reg2_seq.sv(39) @ 651: reporter@@reg2_seq_h [REG2 TEST] FRONTDOOR read PASSED with value: 'haa
# UVM_INFO reg2_seq.sv(45) @ 651: reporter@@reg2_seq_h [REG2 SEQ] BACKDOOR RDATA Value : 55
# UVM_INFO reg2_seq.sv(49) @ 651: reporter@@reg2_seq_h [REG2 TEST] BACKDOOR read PASSED with value: 'h55
# UVM_INFO my_driver.sv(70) @ 690: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 32 ADDR : 8
# UVM_INFO my_scoreboard.sv(51) @ 711: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 750: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 85 ADDR : 8
# UVM_INFO my_scoreboard.sv(51) @ 771: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 810: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 8

```

Figure 9: Execution Log 3: REG2 Sequence Terminal Output

```
# UVM_INFO my_test.sv(76) @ 830: uvm_test_top [REG 3 RUN TEST] TEST HERE
# UVM_INFO reg3_seq.sv(21) @ 830: reporter@@reg3_seq_h [REG3 SEQ ] Initial Desired Value : 0
# UVM_INFO reg3_seq.sv(28) @ 830: reporter@@reg3_seq_h [REG3 SEQ] Updated Desired Value : 170
# UVM_INFO my_scoreboard.sv(51) @ 831: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 870: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 12
# UVM_INFO my_scoreboard.sv(51) @ 891: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(84) @ 930: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 12 RDATA : 170
# UVM_INFO my_scoreboard.sv(55) @ 951: uvm_test_top.env.sco [RIGHT DATA] rdata = aa , Expec_rdata = aa
# UVM_INFO reg3_seq.sv(35) @ 951: reporter@@reg3_seq_h [REG3 SEQ] FRONTDOOR RDATA Value : aa
# UVM_INFO reg3_seq.sv(39) @ 951: reporter@@reg3_seq_h [REG3_TEST] FRONTDOOR read PASSED with value: 'haa
# UVM_INFO reg3_seq.sv(45) @ 951: reporter@@reg3_seq_h [REG3 SEQ] BACKDOOR RDATA Value : 55
# UVM_INFO reg3_seq.sv(49) @ 951: reporter@@reg3_seq_h [REG3_TEST] BACKDOOR read PASSED with value: 'h55
# UVM_INFO my_driver.sv(70) @ 990: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 32 ADDR : 12
# UVM_INFO my_scoreboard.sv(51) @ 1011: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 1050: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 85 ADDR : 12
# UVM_INFO my_scoreboard.sv(51) @ 1071: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 1110: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 12
```

Figure 10: Execution Log 4: REG3 Sequence Terminal Output

```
# UVM_INFO my_test.sv(80) @ 1130: uvm_test_top [REG 4 RUN TEST] TEST HERE
# UVM_INFO reg4_seq.sv(21) @ 1130: reporter@@reg4_seq_h [REG4 SEQ ] Initial Desired Value : 0
# UVM_INFO reg4_seq.sv(28) @ 1130: reporter@@reg4_seq_h [REG4 SEQ] Updated Desired Value : 170
# UVM_INFO my_scoreboard.sv(51) @ 1131: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 1170: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 16
# UVM_INFO my_scoreboard.sv(51) @ 1191: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(84) @ 1230: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 16 RDATA : 170
# UVM_INFO my_scoreboard.sv(55) @ 1251: uvm_test_top.env.sco [RIGHT DATA] rdata = aa , Expec_rdata = aa
# UVM_INFO reg4_seq.sv(35) @ 1251: reporter@@reg4_seq_h [REG4 SEQ] FRONTDOOR RDATA Value : aa
# UVM_INFO reg4_seq.sv(39) @ 1251: reporter@@reg4_seq_h [REG4_TEST] FRONTDOOR read PASSED with value: 'haa
# UVM_INFO reg4_seq.sv(45) @ 1251: reporter@@reg4_seq_h [REG4 SEQ] BACKDOOR RDATA Value : 55
# UVM_INFO reg4_seq.sv(49) @ 1251: reporter@@reg4_seq_h [REG4_TEST] BACKDOOR read PASSED with value: 'h55
# UVM_INFO my_driver.sv(70) @ 1290: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 32 ADDR : 16
# UVM_INFO my_scoreboard.sv(51) @ 1311: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 1350: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 85 ADDR : 16
# UVM_INFO my_scoreboard.sv(51) @ 1371: uvm_test_top.env.sco [my_scoreboard] SCO Write DATA
# UVM_INFO my_driver.sv(70) @ 1410: uvm_test_top.env.agent.driver [DRV] Mode : Write WDATA : 170 ADDR : 16
# UVM_INFO verilog_src/uvm-1.ld/src/base/uvm_objection.svh(1267) @ 1430: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
#
```

Figure 11: Execution Log 5: REG4 Sequence Terminal Output & Test Completion

```

# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    94
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [CNTRL RUN TEST]    1
# [CNTRL SEQ]         4
# [CNTRL_TEST]        2
# [DRV]              24
# [MY_AGENT]          1
# [MY_DRIVER]         1
# [MY_ENV]            1
# [MY_MONITOR]        1
# [MY_SEQUENCER]      1
# [MY_TEST]           1
# [Questa UVM]        2
# [REG 1 RUN TEST]    1
# [REG 2 RUN TEST]    1
# [REG 3 RUN TEST]    1
# [REG 4 RUN TEST]    1
# [REG1 SEQ]          3
# [REG1 SEQ ]         1
# [REG1_TEST]         2
# [REG2 SEQ]          3
# [REG2 SEQ ]         1
# [REG2_TEST]         2
# [REG3 SEQ]          3
# [REG3 SEQ ]         1
# [REG3_TEST]         2

```

Figure 12: Execution Log 6: UVM Report Summary — Severity Counts and Message IDs (Part 1)

```

# [REG3 SEQ]          3
# [REG3 SEQ ]         1
# [REG3_TEST]         2
# [REG4 SEQ]          3
# [REG4 SEQ ]         1
# [REG4_TEST]         2
# [RIGHT DATA]       5
# [RNTST]             1
# [TEST_DONE]         1
# [UVMTOP]            1
# [my_scoreboard]     19
# ** Note: $finish : C:/questasim64_2021.1/win64/.../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 1430 ns Iteration: 61 Instance: /top
# 1

```

Figure 13: Execution Log 7: UVM Report Summary — Zero Errors/Fatals and Simulation Finish at 1430 ns (Part 2)

4.3 Functional Coverage Analysis

The screenshot shows a table of functional coverage data for various register covergroups. All entries show 100.00% coverage, meeting the goal of 100. The status is 'Full toggle coverage' for all, indicated by green checkmarks. The 'Included' column shows 'auto(1)' for all entries.

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_i
/regs_pkg/reg1_re...		100.00%						
TYPE reg1_co...		100.00%	100	100.00...		✓	auto(1)	
CVP reg1_...		100.00%	100	100.00...		✓		
/regs_pkg/cntrl_re...		100.00%						
TYPE cntrl_co...		100.00%	100	100.00...		✓	auto(1)	
/regs_pkg/reg2_re...		100.00%						
TYPE reg2_co...		100.00%	100	100.00...		✓	auto(1)	
/regs_pkg/reg3_re...		100.00%						
TYPE reg3_co...		100.00%	100	100.00...		✓	auto(1)	
/regs_pkg/reg4_re...		100.00%						
TYPE reg4_co...		100.00%	100	100.00...		✓	auto(1)	

Figure 14: QuestaSim Functional Coverage Report: 100% Coverage Attained Across All Register Covergroups

Covergroup / Metric	Coverage Goal	Attained	Status
cntrl_cov (cntrl_reg)	100%	100.00%	Full toggle coverage on control bits [3:0] (bins 0 & 1)
reg1_cov (reg1_reg)	100%	100.00%	Lower [0:63], mid [64:127], and high [128:255] bins hit
reg2_cov (reg2_reg)	100%	100.00%	Lower [0:63], mid [64:127], and high [128:255] bins hit
reg3_cov (reg3_reg)	100%	100.00%	Lower [0:63], mid [64:127], and high [128:255] bins hit
reg4_cov (reg4_reg)	100%	100.00%	Lower [0:63], mid [64:127], and high [128:255] bins hit

Table 2: RAL Verification Functional Coverage Breakdown